10/20: Lecture 16. Savitch's Theorem

1 Review: Space Complexity Classes

As a reminder, we have so far defined the following two space complexity classes:

$$\text{SPACE}[S(n)] = \{ \text{ problems solvable in } c \cdot S(n) \text{ working space } \}$$
$$\text{NSPACE}[S(n)] = \{ \text{ problems solvable in } c \cdot S(n) \text{ working space on a DTM} \}$$

Moreover, for polynomial $S(n)$, we proved that

$$\text{NSPACE}[S(n)] \subseteq \text{TIME}[2^{O(S(n))}]$$

using a configuration graph, i.e. at worst, we have an exponential upper bound for NSPACE.

2 Space Hierarchy Theorem

Definition. Suppose f, g are "space-constructable" and $f(n) \ll g(n)$ for sufficiently large n. Then,

$$\text{SPACE}[f(n)] \subsetneq \text{SPACE}[g(n)].$$

2.1 Known and Open Questions

By the (Nondeterministic) Space Hierarchy Theorem, we **know** that:

$$\text{L} \subsetneq \text{PSPACE}$$
$$\text{NL} \subsetneq \text{NPSPACE}$$

Though we strongly believe these, we have **not yet proven** that:

$$\text{L} \subsetneq \text{NL}$$
$$\text{NL} \subsetneq \text{P}$$
$$\text{P} \subsetneq \text{PSPACE}$$

3 Configuration Graphs

3.1 Review: Nondeterministic Space and Graphs

We discovered last lecture that

$$\text{nondeterministic space-bounded computation} \equiv \text{directed } st\text{-connectivity}.$$

As a result, we defined a **configuration graph** as:

- Vertices: Configurations of N running on input x, with size $2^{O(S(n))}$
- Edges: Valid (possible) transitions between configurations according to the transition function δ_N (i.e. $(C, C') \in E \Leftrightarrow \exists$ transition according to δ_N s.t. C yields C'), with size $2^{O(S(n))}$

With this, we want to answer the question: is there a path from the starting configuration C_{start} to the accepting configuration C_{acc}?

- More concretely, how do we solve directed st-connectivity using nondeterminism while remembering only $O(1)$ vertex names and also able to compute $O(1)$ edge relations in $S(n)$ space?

3.2 Nondeterministic *st*-Connectivity

Algorithm. Given G, s, and t:

```
1: u ← s
2: for i = 1, ..., |V| do
3:     guess v ∈ V
4:     if (u, v) ∉ E then
5:         return REJ
6:     else if v = t then
7:         return ACC
8:     end if
9:     u ← v
10: end for
11: return REJ
```

Correctness.

- **Completeness.** If there exists a path from s to t, then there exists a simple path of length at most $|V|-1$. The algorithm can nondeterministically guess each successive vertex along this path, passing every edge check and eventually accepting when $v = t$.
- **Soundness.** If there is no path from s to t, then every possible sequence of guessed vertices must either encounter an invalid edge or fail to reach t within $|V|$ iterations. Therefore, every computation branch rejects.

Space Complexity.

- We store $O(1)$ vertex names, each requiring $O(\log |V|)$ bits.
- The loop counter i requires $O(\log |V|)$ bits.
- Each iteration performs one edge check which requires $O(1)$ additional space for an explicitly represented graph with $O(1)$ edge-lookup.

Therefore, the total working space is $O(\log |V|)$, which is $O(1)$ in terms of $|V|$.

3.3 Simulating NSPACE Using SPACE

Exhaustive enumeration. If we have a nondeterministic algorithm, we can naively simulate each branch by explicitly enumerating all possible nondeterministic choices (i.e. listing all possible guesses for $v \in V$) until we find an accepting path. However, storing all of the configurations we have encountered result in an upper bound of

$$\text{NSPACE}[S(n)] \subseteq \text{TIME}[2^{S(n)}] \subseteq \text{SPACE}[2^{S(n)}].$$

4 Savitch's Theorem

Claim.

$$\text{NSPACE}[S(n)] \subseteq \text{SPACE}[S(n)^2].$$

Corollary.

$$\text{NPSPACE} = \text{PSPACE}.$$

Algorithm Idea. Middle First Search

- We are going to use a recursive deterministic algorithm to solve the *st*-connectivity problem.
- If there is an *st*-path of length $|V| = 2^{S(n)}$, then there is some midpoint vertex m we can choose s.t. the path lengths from $s \to m$ and from $m \to s$ are each at most $\frac{|V|}{2}$.
- Guess $m \in V$, check that there is an sm-path and an mt-path. If both paths exist, ACC.

Algorithm. MFS(G, s, t, k)

if $k = 1$ **then**
  if $s = t$ or $(s, t) \in E$ **then**
    return ACC
  end if
end if
for each $m \in V$ **do**:
  if MFS$(G, s, m, k/2)$ and MFS$(G, m, t, k/2)$ **then**
    return ACC
  end if
end for
return REJ

Correctness. (Induction on $\log k$, where k = path length)

- **Base case.** Suppose $k = 1$. Then, MFS accepts iff $s = t$ or s and t are connected by an edge. so the algorithm is correct.
- **Inductive step.** Assume that MFS is correct for path lengths at most $k/2$.
  - If there exists an st-path of length at most k, then there exists some midpoint m s.t. both subpaths have length at most $k/2$. By the IH, MFS$(G, s, m, k/2)$ and MFS$(G, m, t, k/2)$ both accept, so MFS accepts.
  - Conversely, if MFS accepts, it must have found some midpoint m for which both recursive calls accept. By the IH, both subpaths exist, so an st-path of length at most k must exist.

Therefore, MFS is correct by induction on $\log k$.

Space Complexity. Consider the recursion tree.

- **Space per level.** We store s, t, m, k. Each level uses $O(\log k) = O(S(n))$ space.
- **Number of levels.** We start with $k = 2^{S(n)}$ and halve it each time, until $k = 1$, which takes $O(\log k) = \log(2^{O(S(n))}) = O(S(n))$ levels.
- **Backtracking.** We must remember which recursive call and midpoint choice we are currently exploring at each level, so that we can backtrack and try the next midpoint. However, we only store one root-to-leaf path at a time, rather than the entire recursion tree.

Therefore, the total space complexity is $O(S(n)) \cdot O(S(n)) = O(S(n)^2)$.